多颜色识别

多颜色识别
- 例程实验效果简介
- 代码讲解
  - 完整代码
  - 代码结构

例程实验效果简介

本节例程代码位于：【源码汇总 / 05.Color / 03.multi_color_recognition.py】

我们用CanMV IDE打开例程代码，将K230用USB连接到电脑上

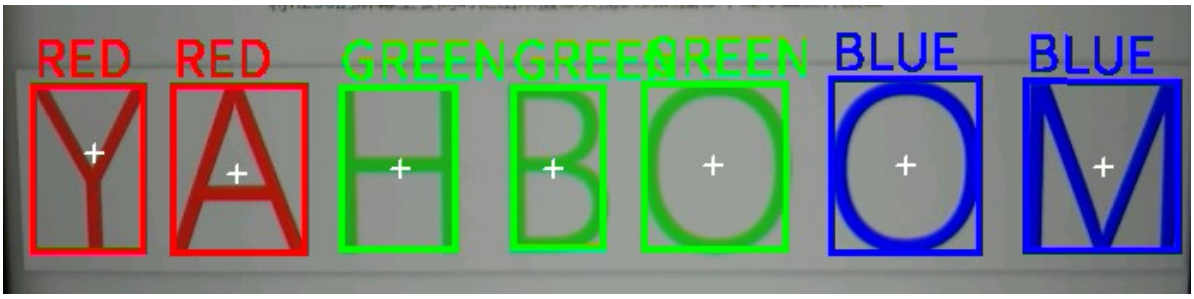
打开例程代码，找到THRESHOLDS数组，在数组中找到你想追踪/识别的颜色的下标

添加方法我们可以看前一节的 **【添加自定义颜色】** 部分



点击CanMV IDE左下角的运行按钮，

将K230的屏幕里会同时框出来摄像头捕获到的图像中红绿蓝三种颜色



代码讲解

本节我们要用的的外设主要是摄像头模块

线段检测由 K230中的 find_blobs() 方法实现，该方法属于image模块

完整代码

```
import time, os, sys
from media.sensor import *
from media.display import *
from media.media import *
```

```
# 显示参数 / Display parameters
DISPLAY_WIDTH = 800    # LCD显示宽度 / LCD display width
DISPLAY_HEIGHT = 480   # LCD显示高度 / LCD display height

# 颜色阈值(LAB色彩空间) / Color thresholds (LAB color space)
# (L Min, L Max, A Min, A Max, B Min, B Max)
COLOR_THRESHOLDS = [
    (0, 66, 7, 127, 3, 127),    # 红色阈值 / Red threshold
    (42, 100, -128, -17, 6, 66),     # 绿色阈值 / Green threshold
    (43, 99, -43, -4, -56, -7),       # 蓝色阈值 / Blue threshold
]

# 显示颜色定义 / Display color definitions
DRAW_COLORS = [(255,0,0), (0,255,0), (0,0,255)]  # RGB颜色 / RGB colors
COLOR_LABELS = ['RED', 'GREEN', 'BLUE']           # 颜色标签 / Color labels

def init_sensor():
    """初始化摄像头 / Initialize camera sensor"""
    sensor = Sensor()
    sensor.reset()
    sensor.set_framesize(width=DISPLAY_WIDTH, height=DISPLAY_HEIGHT)
    sensor.set_pixformat(Sensor.RGB565)
    return sensor

def init_display():
    """初始化显示 / Initialize display"""
    Display.init(Display.ST7701, to_ide=True)
    MediaManager.init()

def process_blobs(img, threshold_idx):
    """处理颜色区块检测 / Process color blob detection"""
    blobs = img.find_blobs([COLOR_THRESHOLDS[threshold_idx]])
    if blobs:
        for blob in blobs:
            # 绘制检测框和标记 / Draw detection box and markers
            img.draw_rectangle(blob[0:4], thickness=4,
color=DRAW_COLORS[threshold_idx])
            img.draw_cross(blob[5], blob[6], thickness=2)
            img.draw_string_advanced(blob[0], blob[1]-35, 30,
                                     COLOR_LABELS[threshold_idx],
                                     color=DRAW_COLORS[threshold_idx])

def main():
    try:
        # 初始化设备 / Initialize devices
        sensor = init_sensor()
        init_display()
        sensor.run()

        clock = time.clock()

        while True:
            clock.tick()

            # 捕获图像 / Capture image
            img = sensor.snapshot()

            # 检测三种颜色 / Detect three colors
```

```
            for i in range(3):
                process_blobs(img, i)

            # 显示FPS / Display FPS
            fps_text = f'FPS: {clock.fps():.3f}'
            img.draw_string_advanced(0, 0, 30, fps_text, color=(255, 255, 255))

            # 显示图像并打印FPS / Display image and print FPS
            Display.show_image(img)
            print(clock.fps())

    except KeyboardInterrupt as e:
        print("用户中断 / User interrupted: ", e)
    except Exception as e:
        print(f"发生错误 / Error occurred: {e}")
    finally:
        # 清理资源 / Cleanup resources
        if 'sensor' in locals() and isinstance(sensor, Sensor):
            sensor.stop()
        Display.deinit()
        MediaManager.deinit()

if __name__ == "__main__":
    main()
```

代码结构

基本设置：

- 设定640×480分辨率的显示屏
- 使用LAB色彩空间定义了红、绿、蓝三种颜色的检测阈值
- DRAW_COLORS和COLOR_LABELS数组用于绘制标记和显示颜色标签

主要改进：

- 同时检测三种颜色
- 为检测到的物体添加颜色标签文字标注
- 简化了颜色处理逻辑

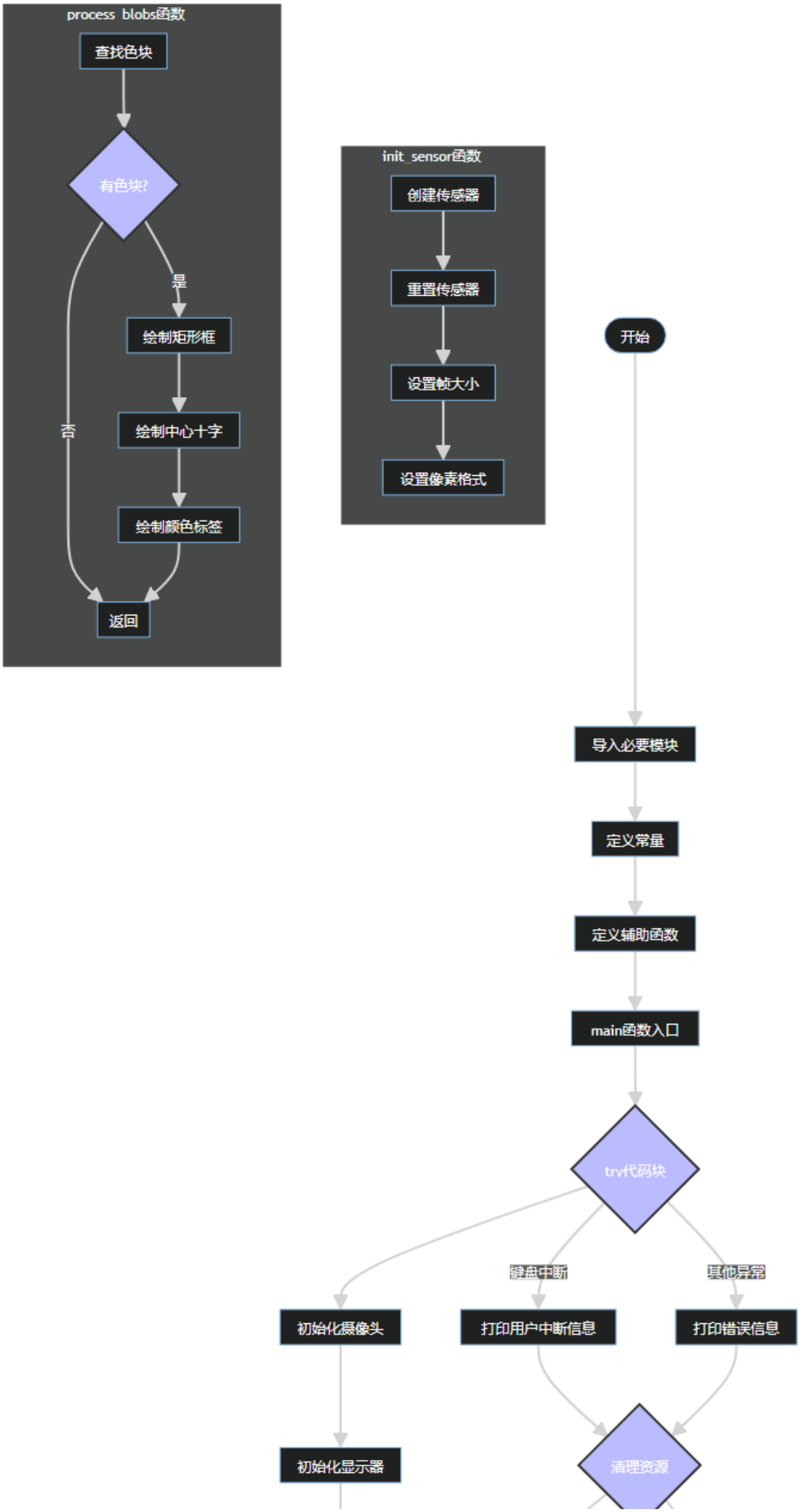
工作流程：

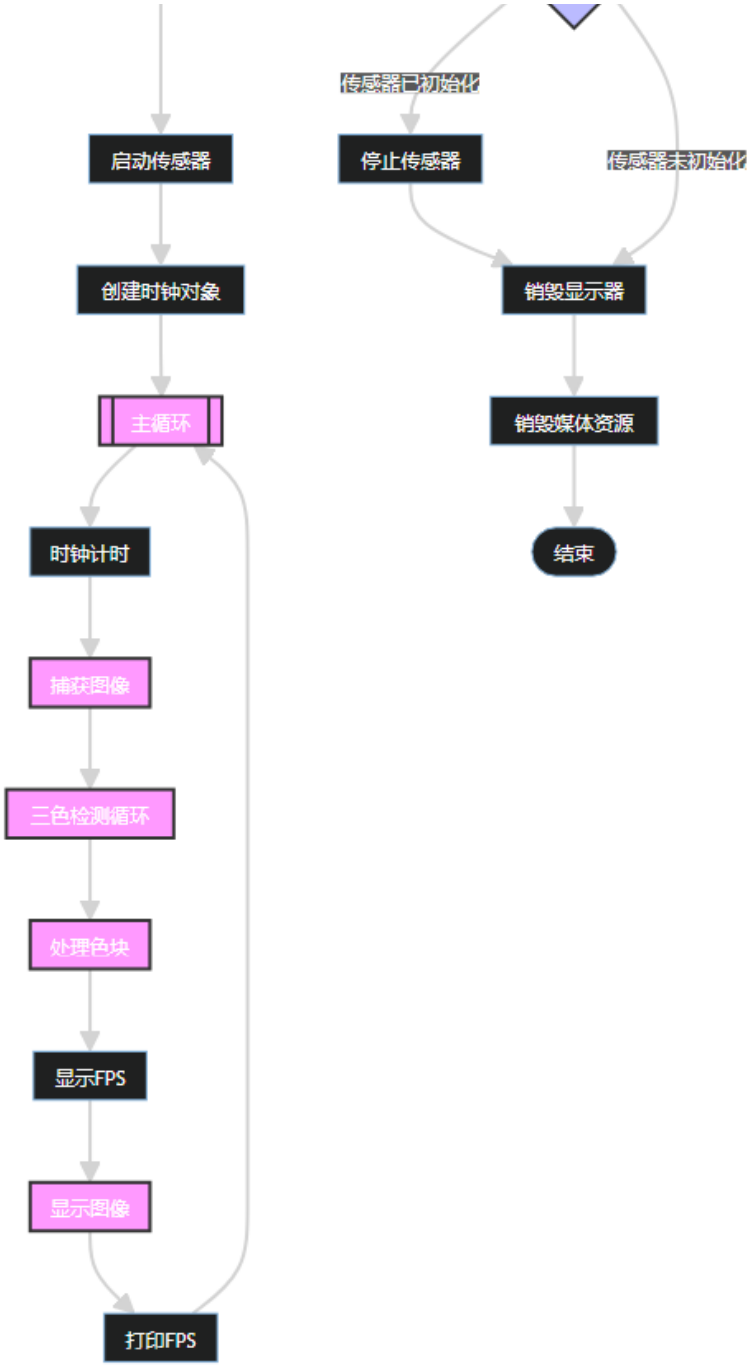
- 初始化摄像头和显示屏
- 进入主循环后：
    - 持续捕获图像
    - 依次检测三种颜色
    - 为每个检测到的物体绘制矩形框、十字标记和颜色标签
    - 显示帧率信息

异常处理：

- 保留了完整的异常处理机制
- 确保程序结束时正确释放硬件资源

详细流程图如下：


process_blobs函数
查找色块
有色块?
是
否
绘制矩形框
绘制中心十字
绘制颜色标签
返回
init_sensor函数
创建传感器
重置传感器
设置帧大小
设置像素格式
开始
导入必要模块
定义常量
定义辅助函数
main函数入口
try代码块
键盘中断
其他异常
初始化摄像头
打印用户中断信息
打印错误信息
初始化显示器
清理资源


传感器已初始化
启动传感器
停止传感器
传感器未初始化
创建时钟对象
销毁显示器
主循环
销毁媒体资源
时钟计时
结束
捕获图像
三色检测循环
处理色块
显示FPS
显示图像
打印FPS